

Response to Reviewers

Manuscript submitted to *Technologies*
Pre-Processing Modeling Artefacts for Graph Neural Networks: A DSL for Model Encoding

Zahra Rajaei, Shekoufeh Kolahdouz-Rahimi, Massimo Tisi

We thank the Editor and the Reviewers for their careful reading of the manuscript and their constructive comments. We have revised the manuscript to address every point raised. In the revised manuscript, all text that was added or substantially changed is typeset in blue to ease re-review. The table below lists each comment, our response, and the corresponding change in the manuscript (with section/table references to the revised version).

Note on the updated evaluation

The revised evaluation is a documented measurement, and the numbers favour the tool on the questions it asks. One configuration reproduces every published graph we checked, by a node bijection that preserves every typed edge: 500/500 Ecore, 500/500 RDS and 369/369 Yakindu. That certificate is more complete than re-running the authors’ Java encoder, which matches 492 of the 500 Ecore graphs. On the same 500 Ecore models, ten fresh processes, every MoEGL run is faster than every Java run (1.11 ± 0.02 s versus 1.48 ± 0.10 s), and the peak working set is lower (153.5 ± 0.2 MB versus 236.6 ± 10.1 MB). The per-task specification is 14–29 lines of YAML; the shared tool, counted once, is 655 lines. Relation tokens raise balanced accuracy from 0.808 ± 0.024 to 0.824 ± 0.025 (paired +0.016, $p = 0.017$, better on 8 of 10 folds), level with the best published name classifier (0.825) and above the published GNN (0.808). All of these figures are produced by the scripts in the replication package (<https://github.com/Z-Rajaei/MoEGL>). An earlier PyEcore loader took 7.98 s and is not the timed tool.

Reviewer 1

The opening summary describes the submitted manuscript. In the revised text the timed loader reads the models with lxml. An earlier PyEcore loader took 7.98 s and is not the timed tool. The pretrained name vectors are WordE4MDE. The edge-preserving certificate covers the graphs published by Lopez et al.: 500/500 Ecore, 500/500 RDS and 369/369 Yakindu. Rahimi et al. published no typed-graph corpus, and the Miranda comparison is the printed 7-node listing. The timed comparison is against their Java encoder, 1.11 ± 0.02 s versus 1.48 ± 0.10 s, on the 500 Ecore models. The two additions beyond the 2021 workshop DSL are heterogeneous graphs and attribute-value encodings. The five recommendations are answered in R1.1–R1.5.

#	Reviewer comment	Response	Changes in the manuscript
---	------------------	----------	---------------------------

#	Reviewer comment	Response	Changes in the manuscript
R1.1	Either perform a formal user study (participants, tasks, SUS scores, statistical analysis) to support the usability claims, or reduce the claims in Sections 2.2, 6.5.1 and 8 to the authors' qualitative observations.	The evaluation now stands on the three questions the article asks, and on the measurements that answer them: the edge-preserving certificate, the timed comparison with the Java encoder, and the classification result. The earlier sentences that called the language easy, or an ideal balance of usability and expressiveness, are removed. A practitioner study is future work.	Section 2.1; Section 6.5 (design properties); the design-aims paragraph in Section 5; Section 8.

R1.2	Extend Table 2 to a full evaluation. State the hardware, the dataset size, the model complexity, the distribution of runtimes over repeated executions (mean $\pm$ standard deviation), the memory requirements, and exactly which program the bespoke column is.	Hardware, dataset size, model size and the ten-run distribution were already stated, and the bespoke column is now named. The machine is an AMD Ryzen 9 4900H with 47 GB of installed RAM, Windows 11, OpenJDK 21.0.3 and Python 3.10.9. The timed dataset is the 500 Ecore metamodels of Experiment 1 (21,938 objects). Table 1 gives the size of those models: 61.4 objects and 159.6 references become graphs of 43.9 nodes and 120.5 edges. Times are the mean $\pm$ sample standard deviation over ten fresh processes. The bespoke encoder is the Java implementation of Lopez et al., executed by <code>TimeLopezAll</code> on the Ecore set. That driver performs the steps of their <code>GenerateRealGraphsEcore</code> program (resource loading, graph construction, JSON serialisation) and does not filter or sample the corpus, because the released directory is already the set of 500. Memory of the encoding process had not been measured. It is now, on the same two commands and the same 500 models, in a separate session of ten fresh processes. The peak working set of the worker is 236.6 ± 10.1 MB for <code>java.exe</code> and 153.5 ± 0.2 MB for MoEGL’s <code>python.exe</code>. The peak committed memory is 957.0 ± 10.8 MB and 624.6 ± 0.3 MB. One MB is 2^{20} bytes. The 47 GB figure is the installed RAM, not the memory either process used.	Section 6.5 (measurement protocol, Table 2, results, threats to validity); Table 1 for model size.
------	---	--	---

R1.3	Run at least one downstream learning task that compares a homogeneous graph with a heterogeneous graph produced by MoEGL, and report accuracy and training convergence.	Section 6.6 is that task, on the near-duplicate ModelSet split (2,074 Ecore models, 48 categories). Both graphs come from MoEGL on the same models, and both models see the same WordE4MDE name vector on each node. The homogeneous model is a GIN, which ignores edge types. The heterogeneous model is an RGCN on the nine edge types. Balanced accuracy on the ten test folds is 0.770 ± 0.029 (GIN) and 0.795 ± 0.036 (RGCN). The RGCN is higher on 7 of the 10 folds. The mean paired difference is 0.025 (standard deviation 0.037; paired $t = 2.14$, two-sided $p = 0.061$). The score sits next to the best published name classifier (0.825). The same protocol was run again with the validation score stored after every epoch. Training lasts at most 30 epochs and stops after 6 epochs without an improvement. The selected epoch is 16.4 ± 3.8 for the GIN and 15.2 ± 4.7 for the RGCN; the runs last 22.4 ± 3.8 and 21.2 ± 4.7 epochs. Validation balanced accuracy at the selected epoch is 0.810 ± 0.042 and 0.822 ± 0.042. On the first 15 epochs, which every fold reached, the mean validation score of the RGCN is higher at every epoch, starting from 0.398 against 0.165 after epoch 1. The typed graph raises the score training reaches, at a similar number of epochs. The test scores of this run match Table 3.	Section 6.6; Table 3.
------	---	--	-----------------------

R1.4	The same material is written twice. Either remove lines 65–74 of the introduction, or bring Section 2 into the introduction. Shorten Section 8, because it retells the narrative already given in Section 6.	Lines 65–74 of the submitted introduction were a second list of research questions, and that list differed from Section 2.2. The list is removed. The introduction now names the three questions in one sentence and points to Section 2.1, which is the only place they are stated in full. The later introduction paragraphs that defined MoEGL, the heterogeneous graphs and the experimental outcome again are reduced to one paragraph. Section 8 no longer repeats the graph counts, the times or the accuracies. It names the experiment that answers each question and points to Sections 6.1, 6.5 and 6.6, where those results are reported. The conclusion points to Section 8 and does not copy the figures.	Introduction; Section 2; Section 8; Conclusion.
------	---	--	--

R1.5	The related work cites nothing from 2025 or 2026.	Five publications from those years are now discussed in the related work. ModelXGlue (Software and Systems Modeling, 2025) separates encoding from training and records that a GNN needs a graph. The WordE4MDE journal paper (Software and Systems Modeling, 24(6):1647–1669, 2025) extends the 2023 vectors with subword units, a modelling BERT, and forum text. Our classification still uses the 2023 skip-gram vectors. Khalilipour et al. (MODELS-C 2025) classify Ecore metamodels with a GNN on a semantic graph and report about 96% accuracy on their 24-category retrieval task with BERT features. That figure is theirs. The category set, the metric and the features differ from Section 6.6, and we do not enter it in Table 3. Chen et al. (MODELS 2025) generate a graph model from text. Ali (TU Wien dissertation, 2026) encodes Ecore, OntoUML and ArchiMate models as knowledge graphs for a GNN. None of the five is a configuration language for the encoding. The measured numbers of this manuscript are unchanged.	Section 7.
------	---	--	------------

Reviewer 2

#	Reviewer comment	Response	Changes in the manuscript
---	------------------	----------	---------------------------

#	Reviewer comment	Response	Changes in the manuscript
R2.1	Table 2 shows MoEGL is faster (2 s vs 4.2 s) and needs less code (10 vs 85 lines), but the paper does not explain how these numbers were obtained. Which dataset was used, on what computer, and how many times was it run? Also, comparing 10 lines of YAML with a full Python script is not fair, because the MoEGL tool itself is also code. The paper also claims better scalability, ease of use and maintainability, but none of these were measured. Please add real measurements or remove these claims.	The previous Table 2 is replaced by a documented measurement. On that measurement every MoEGL run is faster than every Java run, and each task is 14 to 29 lines of YAML against a few hundred lines of encoder code. (1) <i>Protocol</i>. A new subsection states the dataset (the 500 real Ecore metamodels of Experiment 1, released by Lopez et al. with the graphs their encoder produced), the two implementations (their Java encoder, unchanged, and MoEGL), what is timed (in-process wall time of the encoding loop, and total process time), the repetitions ($N = 10$ fresh processes, mean $\pm$ sample standard deviation), and the machine (AMD Ryzen 9 4900H, 47 GB RAM, Windows 11, OpenJDK 21.0.3, Python 3.10.9, lxml, NetworkX 3.4.2). (2) <i>Time</i>. The current loader, timed with Java in one session ($N = 10$ fresh processes), takes 1.11 ± 0.02 s against 1.48 ± 0.10 s. Stage 1 is 0.67 s and Stage 2 is 0.45 s. Every MoEGL run in that session was faster than every Java run. Re-encoding from the cache takes 1.16 ± 0.33 s. An earlier PyEcore measurement of 7.98 ± 0.11 s is kept as history and is not the number in the table. (3) <i>Code size, counted on both sides</i>. <code>cloc</code> counts the code written once and the code written per task. MoEGL itself is 655 lines of Python, written once and shared. The bespoke encoders are 409 (+190 driver) lines for Lopez et al., 249 for Rahimi et al. and 193 for Miranda et al. Each of those three tasks is now 29, 14 or 19 lines of YAML.	Section 6.5 (protocol and Table 2); Section 6.1.

		(4) Scalability, ease of use, maintainability and extensibility are stated as design properties of that architecture: the cache is built once, a task is a short YAML file, and a new value encoder is one function. Threats to validity (construct, internal, external, reliability) accompany the measurements. (5) Table 1 was recomputed from the runs: 500 Ecore models (61.4 objects, 159.6 references) become graphs of 43.9 nodes and 120.5 edges; Experiment 2 has 11 CarWash models; Experiment 3 is the one statecharts file shipped with the prototype. (6) The timed loader certifies 500/500 Ecore, 500/500 RDS and 369/369 Yakindu graphs by a bijection that preserves every typed edge. An earlier PyEcore loader matched 495 of 500 Ecore graphs. The authors’ Java code matches 492 of 500, because eight released files reference themselves under their original name. The Ecore graphs after the final loader edit were identical to the certified graphs. Scripts and raw results are in the replication package.	Sections 6.2 and 6.3; Tables 1 and 2; Section 6.5 (results, design properties, threats); Section 8; Data Availability.
R2.2	The paper says users found MoEGL easy to use (line 737 and section 8), but no user study is described. We don’t know who the users were, what they did, or how it was measured. Please add a small user study or remove this claim. The paper also says MoEGL works well for different MDE tasks, but no MDE task (like classification) was actually done.	The manuscript now reports what was measured. One language expresses the three published encoders, and Section 6.6 adds the learning task the comment asked for: domain classification, where relation tokens raise balanced accuracy from 0.808 to 0.824. The sentence that users found MoEGL easy, and the sentence that it works well for every MDE task, are removed. A practitioner study is future work.	Section 2.1; Section 6.5.3; Section 8; Conclusion.

R2.3	The research questions in the introduction (lines 65–71) and in section 2.2 are different. Please use one set of questions and show clearly which experiment answers each one. The introduction also uses “model transformation” when the paper is about “model encoding”, and these are not the same thing.	The manuscript now has one set of three questions, stated in the introduction and repeated in Section 2.1, and each question names the experiment that answers it. RQ1 (fidelity) is answered by Experiment 1: an edge-preserving bijection matches 500/500 Ecore, 500/500 RDS and 369/369 Yakindu published graphs. RQ2 (cost) is answered by Section 6.5: 14–29 lines of YAML per task, and a first run faster than the Java encoder on every one of ten processes (1.11 ± 0.02 s versus 1.48 ± 0.10 s). RQ3 (effect of heterogeneous graphs and attribute encoding) is answered by the classification experiment in Section 6.6. Where the questions used to say “model transformation”, they now say “model encoding”. Background sentences that describe MDE in general still use “model transformation” for that development activity.	Section 1; Section 2.1; opening of Section 6; Section 8.
------	---	---	---

R2.4	The two main new features of this paper are heterogeneous graphs and attribute encoding, but the experiments did not test either of them (lines 665–669 say attributes were not used). I suggest adding one learning experiment, for example classifying models with a GNN, to show whether heterogeneous graphs or different attribute encodings give better results.	We added that experiment. Experiments 1–3 still encode no attribute values, because the encoders they replicate do not. The new section turns the features on for domain classification of Ecore metamodels from ModelSet, after removing near-duplicates (Jaccard at least 0.8 within a category) and categories with fewer than ten models: 2,074 models, 48 categories. Lopez et al. (MODELS 2022) use 2,068 models and 48 categories; our file list is not theirs. Balanced accuracy, stratified 5-fold cross-validation, two seeds; 15% of each training category is validation; the test fold is scored once. A linear SVM on element names scores 0.808 ± 0.024. Names plus MoEGL relation tokens score 0.824 ± 0.025 (paired $+0.016$, $t = 2.93$, $p = 0.017$, better on 8 of 10 folds). With WordE4MDE name vectors held fixed, a homogeneous GIN scores 0.770 ± 0.029 and a heterogeneous RGCN 0.795 ± 0.036. Numeric structural attributes lower the RGCN to 0.782 ± 0.040. Lopez et al. report 0.825 (FFNN on TF-IDF), 0.816 (SVM on TF-IDF) and 0.808 (GNN) on their no-duplicate split. Our relational SVM reaches 0.824, level with their best and above their SVM and their GNN. The file lists differ, so the comparison is of published figures. Heterogeneous edges raise the GNN from 0.770 ± 0.029 to 0.795 ± 0.036, higher on 7 of 10 folds. Numeric attribute flags score 0.782 ± 0.040. WordE4MDE, in R2.6, matches TF-IDF names at 0.808 ± 0.024 and is the encoder we provide when a dataset is too small to fit word2vec.	Section 6.6 and Table 3; Section 2.1 (RQ3); Section 8; Threats to validity.
------	---	--	--

R2.5	The checks that MoEGL gives the same graphs as previous work are weak. In Exp. 1, <code>is_isomorphic</code> only compares the shape of the graphs, not the node and edge types. In Exp. 2, the comparison method is not explained, and in Exp. 3 only one small graph with 7 nodes was compared. Also, the numbers in Table 1 do not match the text, for example Exp. 2 has 10 models in the table but 20 in line 613. Please explain how the graphs were compared and fix the table.	The reviewer is right about the submitted checks, and we replaced them. <i>Experiment 1.</i> The certificate is no longer NetworkX’s default <code>is_isomorphic</code>. Colour refinement starts from the node type, and a bijection is accepted only when every typed edge is preserved. It matches all 500 published Ecore graphs, all 500 RDS graphs and all 369 Yakindu graphs. Re-running the authors’ Java encoder matches 492 of the 500 Ecore graphs, so the certificate is also more complete than that re-run. Eight further Yakindu files have no published graph. <i>Table 1.</i> Recomputed from the runs. Experiment 2 is 11 CarWash models in both the table and the text. The old pair (10 in the table, 20 in the text) was removed. Experiment 3’s row is one file: 101 objects, 100 nodes, 130 edges. <i>Experiment 2.</i> The printed listing is MoEGL’s encoding of one of the eleven models (class as node type, reference name as edge type). Rahimi et al. did not publish a typed graph corpus. Their encoder builds a matrix and then drops direction and edge labels before NetGAN, so we do not report an isomorphism score against that matrix. <i>Experiment 3.</i> The printed listing is the 7-node sample of Miranda et al. (3 Transition, 4 State, 6 typed edges), checked entry by entry. The 100-node row of Table 1 is a different file from their prototype and has no published reference graph. The previous text treated these as one comparison; it no longer does.	Section 6.1 (comparison paragraph); Section 6.2; Section 6.3; Table 1.
------	---	---	---

R2.6	The word2vec values in Listing 4 are all very small (around ± 0.01), which suggests the model was not really trained, because the text data from the models is too small. Please explain how word2vec was trained, how model names are split into words, and what happens with unknown words. Using a pre-trained model like WordE4MDE may work better.	The diagnosis is correct, and we removed the vector. On the running example the local skip-gram model has only a handful of names, so its coordinates stay near the random initialisation (scale $0.5/d$). The revised section states the procedure. Names are split on non-alphanumeric characters and on camelCase boundaries (<code>hasCustomer</code> becomes <code>has</code> and <code>customer</code>). Skip-gram Word2Vec is fitted once on the tokens of the whole dataset (dimension 64, window 3, minimum count 1, 20 epochs, seed 42). The value’s vector is the mean of its known token vectors. A value with no known token becomes the zero vector. WordE4MDE (López, Durá and Sánchez Cuadrado, MODELS 2023) is now an encoder in the same interface: the authors’ 300-dimensional skip-gram vectors, the same tokeniser, and a zero vector when every token is unknown. On the classification split a linear SVM on the mean WordE4MDE vector of each model scores 0.808 ± 0.024, level with TF-IDF names, and a GIN with those vectors as node features scores 0.770 ± 0.029. It is the encoder the tool offers for a dataset too small to fit word2vec, and on this split it matches the bag-of-names baseline.	Section 5.2 (attribute-value encoding); the running-example graph listing (coordinate dump removed); Section 6.6.
------	---	--	--

R2.7	The YAML listings do not use the same names, for example <code>input</code> versus <code>inputmodels</code>, <code>modelpath</code> versus <code>modelspath</code>, and <code>includeAttributes</code> versus <code>includeAllAttributes</code>. Listing 3 uses the class <code>Transaction</code>, which should be <code>Transition</code>, and it has a syntax error: <code>encoding</code> "one-hot". Please make all listings consistent, give one clear definition of the DSL syntax, and check that all examples actually run.	The listings now use one syntax, and that syntax is a file the tool loads. The keys are <code>input</code>, <code>input.metamodelpath</code>, <code>input.modelspath</code>, <code>output.format</code>, <code>includeAllAttributes</code> and <code>features.encoding</code>. The class in the attribute listing is <code>Transition</code>, and the broken line is <code>encoding: "one-hot"</code>. The schema listing no longer uses leading dashes or the older key names. All four illustrative listings were loaded and executed. The structure listing produces the 8-node graph of Matrix 1 (13 edges, no self-loop). The attribute listing produces the heterogeneous graph now printed (FSM, 3 Transition, 3 Action, 4 State). The metamodel listing encodes <code>name</code> and <code>abstract</code>; <code>eType</code> stays an edge because it is a reference. The experiment listings are the configurations already used for Tables 1 and 2. Paths are relative to the MoEGL package root, under <code>examples/running_fsm</code>.	Section 5 (syntax paragraph and the corrected listings); the running-example output; Section 6.7.
------	---	--	--

R2.8	The paper should explain what MoEGL cannot do. For example, can it turn attributes into nodes, add reverse edges, or add edge features? Can it reproduce tree-based encodings or graph-kernel encodings? A table of which existing encoders MoEGL can and cannot reproduce would help. Also, please explain why the FSM node has a link to itself in Matrix 1.	An attribute stays a value on its element; there is no construct that turns it into a node. A reverse edge is written when the feature records an opposite and that opposite feature is not derived. On the Ecore metamodel used here, <code>EReference.eOpposite</code> records no opposite, so that edge is written only where the file serialises it. There is no key that inserts any further edge. An edge stores the reference name as its type and does not store a feature vector. Table 4 records the boundary: the Lopez, Rahimi and Miranda graph encoders are configurations of MoEGL; the graph kernel of Clarisó and Cabot and the tree-to-sequence encoding of Burgueño et al. are not produced. The 1 on the FSM diagonal was a mistake. The FSM class has no reference to itself. Re-running the listing on the instance drawn in the object diagram gives the matrix now printed: the first row is 0 1 1 1 1 1 1 1, eight nodes, thirteen edges, and a zero diagonal.	Section 6.7 and Table 4; Matrix 1 and the paragraph that introduces it; Section 8.
------	---	--	---

R2.9	The claim that this is the first approach of its kind is too strong. Some related work is missing, such as studies using ModelSet and general tools in PyG and DGL. Some references are wrong. [39] is not about metamodel evolution, the VIATRA paper is cited as Varró in the text but listed under Csertán, and [28] is called Rahimi in one place and Abbas in another.	The three “first” sentences are removed. A 2021 workshop paper already proposed the DSL for homogeneous graphs. This article adds heterogeneous graphs and attribute encodings, and it measures them. ModelSet is now cited where the classification split is defined (López, Cánovas Izquierdo and Sánchez Cuadrado, Software and Systems Modeling, 2022). Related work names PyTorch Geometric and the Deep Graph Library. MoEGL writes NetworkX and PyG heterogeneous graphs. It does not emit DGL. The Kolovos et al. (2008) subsection is removed. That paper detects inconsistencies across heterogeneous models, and MoEGL does not migrate schemas. VIATRA is now cited as Csertán et al.; Varró is a co-author. The CarWash paper is Rahimi et al. in both the experiment and the related work. The first author is Abbas Rahimi.	Introduction; Section 7; Section 6.6 (ModelSet citation); Section 8.
------	--	---	---

R2.10	The paper repeats its contributions three times and uses too much promotional language, such as “significant leap forward” and “transformative potential”. Some figure references are wrong. The template text in Appendix A should be removed, and Section 3 is general background and can be shorter.	The contribution list in Section 2 now points at the three questions, and Section 8 points to the experiment that answers each one, so the figures are reported once, where they are measured. The sentences on a significant leap and on transformative potential are deleted. The results that remain are the certificate, the timing and the classification. The configuration is now tied to the class diagram, not to the architecture figure. The heterogeneous output of the running example is cited as a listing. It is not a figure, and it is not a matrix. Appendix A, including the publisher’s template sentence, is removed. There is no appendix. Section 3 now keeps four short points: what a GNN reads, an EMF model as a typed graph, homogeneous versus heterogeneous graphs, and the fact that the timed loader is lxml. The earlier PyEcore loader is named as history. The running-example figures stay.	Sections 1, 2.1, 3, 4 and 8; the running-example output; removal of Appendix A.
-------	--	---	--

Reviewer 3

#	Reviewer comment	Response	Changes in the manuscript
---	------------------	----------	---------------------------

#	Reviewer comment	Response	Changes in the manuscript
R3.1	The semantics and guarantees of the DSL are only partially defined. In particular: coverage, conflict resolution, identities across cross-resource references, cycles, multi-edges, and edge attributes.	These six points are now stated as the behaviour of the loader and of the adaptation step. Coverage is the include/exclude policy: exclude is applied first, class matching includes supertypes, a dropped class removes its objects and their edges, and a dropped feature removes only that feature. There is no separate coverage score. A name listed in both include and exclude is excluded. Renaming is taken from the most specific class section that defines it. Each object receives an integer identity. A reference is resolved inside the same file by an EMF fragment or an <code>xmi:id</code>. An absolute URI, a <code>platform:</code> or <code>pathmap:</code> URI, and a relative URI whose file exists beside the model produce no object and no edge. A fragment that still resolves after a rename is kept and the URI is recorded as repaired. An unresolved <code>#</code> token is stored as an attribute named by the feature plus Name, such as eTypeName. Containment is walked as a tree. A cycle that the file already contains is copied when both ends and the feature are kept. The one added edge is the opposite named by the metamodel, when that opposite feature is not derived. EReference.eOpposite records no opposite in the Ecore index used here, so that edge is written only where the file serialises it. The graph is a MultiDiGraph with at most one edge per source, target and feature name. The only edge key is type. Attribute encodings stay on nodes. The measured numbers of the manuscript are unchanged.	Section 5 (semantics of a configuration); Section 6.7.

R3.2	The paper does not address leakage from training word2vec on the full corpus before the split, nor the practice of fitting an embedding inside a learning protocol.	The local word2vec model, and the one-hot vocabulary, are fitted on whatever set of models is passed to the encoder. For a supervised split that call has to be made on the training fold alone; the fitted model is then applied to the validation slice and the test fold, and a token absent from the training vocabulary stays the zero vector. Fitting on the whole corpus before the split would place test names inside the vectors. Encoding a dataset that has no held-out split, such as the timed re-encoding, still uses the fit on the whole set. Table 3 does not use that fit. The TF-IDF vectorizer is fitted on the training indices of each fold. The relation tokens are read from the metamodel. The GNN features are a lookup in the frozen WordE4MDE table, which is not estimated on this split. We have not checked whether ModelSet names occur in the corpus on which WordE4MDE was trained. The balanced accuracies 0.808 and 0.824 are the TF-IDF SVM and the relation-token SVM. The measured numbers are unchanged.	Section 5 (encoding of attribute values); Section 6.6; Section 6.5 (threats to validity).
R3.3	Performance, scalability and memory usage are not examined for extraction to CSV and for reconstructing graphs.	The two writers emit an in-memory NetworkX graph and a PyTorch Geometric heterogeneous graph, which is the encoding the experiments time. On the 500 Ecore models, ten fresh processes, MoEGL takes 1.11 ± 0.02 s against 1.48 ± 0.10 s for the Java encoder, and the peak working set is 153.5 ± 0.2 MB against 236.6 ± 10.1 MB (Table 2). A timing series on corpora larger than this set is future work.	Section 5 (output formats); Section 6.5; Table 2.

R3.4	The paper mentions no usable datasets, tasks, baselines, metrics, or quantitative results (accuracy, runtime saved, code reduction, fidelity checks).	The revised manuscript reports each of these. Fidelity is an edge-preserving node bijection against the graphs published by Lopez et al.: 500/500 Ecore, 500/500 RDS and 369/369 Yakindu (Section 6.1), more complete than re-running their Java encoder (492/500). Cost is in Section 6.5 and Table 2: the same 500 Ecore models (21,938 objects), the Java encoder as the baseline, and wall-clock time and peak memory over ten fresh processes. Every MoEGL run is faster (1.11 ± 0.02 s versus 1.48 ± 0.10 s) and the peak working set is lower (153.5 ± 0.2 MB versus 236.6 ± 10.1 MB). The per-task YAML configurations are 14 to 29 lines; the shared tool is 655 lines and is counted. The learning task is domain classification on the near-duplicate ModelSet split (2,074 models, 48 categories), with balanced accuracy. Names plus relation tokens reach 0.824 ± 0.025, up from a TF-IDF linear SVM at 0.808 ± 0.024, and level with the best published name classifier on the authors' own split (0.825).	Sections 6.1, 6.5 and 6.6; Tables 1–3.
R3.5	A major motivation is the lack of an empirical analysis of heterogeneous versus homogeneous encoding on a downstream task.	Section 6.6 is that analysis. Both graphs are produced by MoEGL from the same models, and both models see the same WordE4MDE name vector on each node. The homogeneous model is a GIN, which ignores edge types. The heterogeneous model is an RGCN on the nine edge types. Balanced accuracy is 0.770 ± 0.029 (GIN) and 0.795 ± 0.036 (RGCN). The RGCN is higher on 7 of the 10 folds. The mean paired difference is 0.025 (standard deviation 0.037; paired $t = 2.14$, two-sided $p = 0.061$). On the validation curve the typed graph is ahead at every one of the first 15 epochs. The score sits next to the best published name classifier (0.825).	Section 6.6; Table 3.

R3.6	There is no user study, task-completion time or qualitative feedback to support usability claims for non-programmers.	The manuscript now rests on the measurements: three published encoders expressed by one language, a faster and lighter run than the Java encoder on all ten processes, and the classification result of Section 6.6. The sentences that presented ease of use as a finding are removed. A practitioner study is future work.	Introduction; Section 2; Section 6.5; Section 8.
R3.7	Does MoEGL allow edge attributes, such as multiplicities and containment flags, and can they be exported as PyG HeteroData edge features? If not, what are the restrictions and plans?	The HeteroData writer stores each relation as <code>edge_index</code> only, under the triple of source type, reference name and target type. The printed heterogeneous graph of the running example shows this: every relation is an <code>edge_index</code> and there is no edge-feature tensor. Multiplicity and the containment flag are attributes of the object that declares the reference. In the Ecore encoding that object is a node, so the values stay on the node. Section 6.6 put those flags on the nodes of an RGCN; balanced accuracy moved from 0.795 ± 0.036 to 0.782 ± 0.040 . No YAML key copies them onto the edge. An edge-feature tensor is not part of the current tool, and we do not add a plan or a date for it.	Section 5 (HeteroData listing); Section 6.7; Table 3.
R3.8	What was the outcome of the scalability tests (number of model elements, peak memory, throughput)? Is the CSV step required or optional, and what are its pros and cons?	Tables 1 and 2 are the measured encoding: 500 models and 21,938 objects, on average 61.4 objects and 159.6 references, encoded as 43.9 nodes and 120.5 edges. Wall-clock time is 1.11 ± 0.02 s, about 2.2 ms per model, and the peak working set is 153.5 ± 0.2 MB, below the Java encoder on the same files. The timed path is the in-memory NetworkX writer. A series at several corpus sizes is future work.	Section 5 (output formats); Section 6.5; Tables 1 and 2.

R3.9	The paper does not cover the robustness, reliability and security of downstream machine-learning systems. Please discuss LM-Fix, the fault-injection jailbreak of quantized language models, and “Faults that fortify”, and add a short paragraph on robustness and reliability.	The three papers are now cited in the threats to validity, each for the threat it studies. LM-Fix (IEEE ICCD 2025) detects and recovers bit-flips inside a language model. Zahran et al. (Great Lakes Symposium on VLSI 2025) jailbreak quantized language models by fault injection into their weights. Omid et al. (arXiv:2608.20572, 2026) measure the adversarial accuracy of CNNs trained on an undervolted GPU. The reliability this article reports is its own: the edge-preserving reproduction of every published graph we checked, and the split protocol of Section 6.6, in which the test fold is scored once.	Section 6.5 (threats to validity).
R3.10	Do the authors intend to use DGL, or to keep on-disk formats such as a PyG InMemoryDataset or Parquet, so that large datasets are easier to share?	The experiments use the two writers the tool provides: an in-memory NetworkX graph and an in-memory PyTorch Geometric heterogeneous graph. That NetworkX path is the one timed in Section 6.5. Related work names the Deep Graph Library as a separate graph-learning library.	Section 5 (output formats); Section 7.
R3.11	How does MoEGL support meta-path construction and schema introspection for heterogeneous GNNs that use explicit meta-relations, such as HAN and MAGNN?	The HeteroData writer emits each relation as the triple of source type, reference name and target type, stored as <code>edge_index</code> . That triple is the schema. The printed graph of the running example shows these triples and no other schema object. A meta-path is not constructed. HAN and MAGNN are not implemented, and they are not part of the classification experiment. The heterogeneous model that was run is the RGCN of Section 6.6, which uses the nine edge types directly.	Section 5 (HeteroData listing); Section 6.6.

R3.12	Does a small user study or practitioner feedback exist, showing that non-experts can use MoEGL and can shorten development time under failure, together with learnability observations?	The evidence we report for a shorter encoding is the line count: 14 to 29 lines of YAML for a task that previously took a few hundred lines of Java or Python, with the shared tool counted separately at 655 lines. The learning task shows that those configurations produce graphs a classifier can use. A practitioner study of learnability is future work.	Introduction; Section 2; Section 6.5; Section 8.
-------	---	--	--
